COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct 14, 2026	**DATE SUBMITTED:**	Oct 21, 2026
NAME:	Bal, Kevin Ranz J. Braganza, Ralph Angelov F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:

ILO A

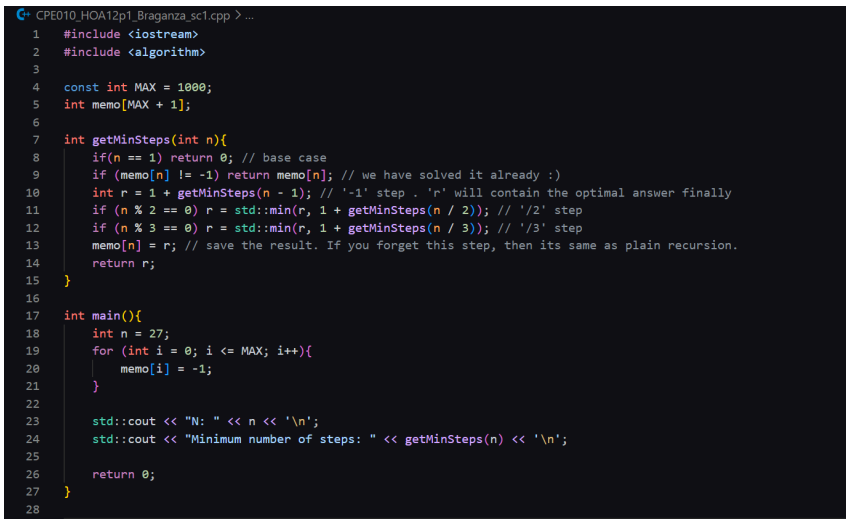
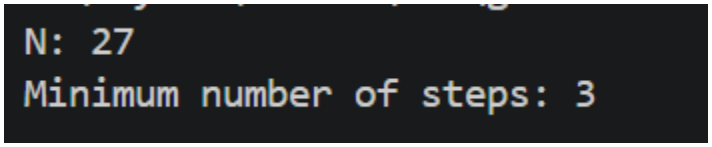
Task Title & Task Objective

For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

Strategy	Algorithm	Analysis
Recursion	**Recursive Power Function**	This function rpower() performs its own method of recursion through rpower(x, n-1) inside of it as a parameter to reach its conditional case of n == 0 and then return the value of 1. For each time this recursion function calling itself, it makes an operation where the exponent of it is being reduced until this recursion function reach its condition and returning its value
Brute Force	**Linear Search Algorithm**	This searching algorithm explores through a list or an array by looking at each item one at a time until it finds what it's looking for or runs out of elements. It does a straightforward check by

		looking at the current value to see if it matches, and if not, it will continue to check and go to the next position until it finds its value or runs out of elements. It uses an index pointer or a loop to track its exact position in the list and stops the whole program when it finds its match or goes through the last value of the list or array.
Backtracking	**Depth-First Search (DFS)**	This searching algorithm explores or traverses and visits each vertex in the graph until it visits everything and reaches its last vertices, it performs a backtracking by going back to the previous vertices that have been visited to find or search more unvisited vertices / nodes before reaching its limit. It uses a Stack STL as a memory of the vertices that have been visited and pops the current node and returns to the previous node to search more unvisited branches of the graph.
Greedy	**Selection Sort**	This sorting algorithm reads the given array and finds the smallest element of it and swaps it to its correct index position. This one doesn't check or read any other position of the array, but only the smallest index value that has been found of the array that will also be sorted.
Divide-and-Conquer	**Merge Sort**	This sorting algorithms performs a Divide-and-Conquer method where searching and getting the midpoint of the array and splitting it into two sub-arrays where those two sub-arrays will be splitted again until it become a small array and will be sorted each of the small array by comparing the value of each where the from to smallest to highest and do the same on the other sub array until both of them will combine again to be a fully sorted array.

Table 12-1. Algorithmic Strategies and Examples

Screenshot of the code	
Output of the code	
Analysis	This memoization algorithm works by checking three options/paths multiple times which are subtracting 1, dividing by 2, or dividing by 3 until the number reaches the base case of 1 and then comparing which option gives the lowest number of steps and saving it in a memory bank/array until it returns the final answer. For the input value of 27, it divides by 3 multiple times going from 27 to 9 then 3 and finally hitting 1, which gives the final output of 3 minimum steps. It takes a very short time to run no matter how large the number is because it checks the array first to see if it already solved it. This still gives a time complexity of O(n) which makes it a lot faster and dependable than plain recursion when handling a large input/number.

```
CPE010_HOA12p1_Braganza_sc1.cpp > ...
#include <iostream>
#include <algorithm>

const int MAX = 1000;
int memo[MAX + 1];

int getMinSteps(int n){
    if(n == 1) return 0; // base case
    if (memo[n] != -1) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps(n - 1); // '-1' step . 'r' will contain the optimal answer finally
    if (n % 2 == 0) r = std::min(r, 1 + getMinSteps(n / 2)); // '/2' step
    if (n % 3 == 0) r = std::min(r, 1 + getMinSteps(n / 3)); // '/3' step
    memo[n] = r; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}

int main(){
    int n = 27;
    for (int i = 0; i <= MAX; i++){
        memo[i] = -1;
    }

    std::cout << "N: " << n << '\n';
    std::cout << "Minimum number of steps: " << getMinSteps(n) << '\n';

    return 0;
}
```

```
N: 27
Minimum number of steps: 3
```

Table 12-2. Memoization Implementation

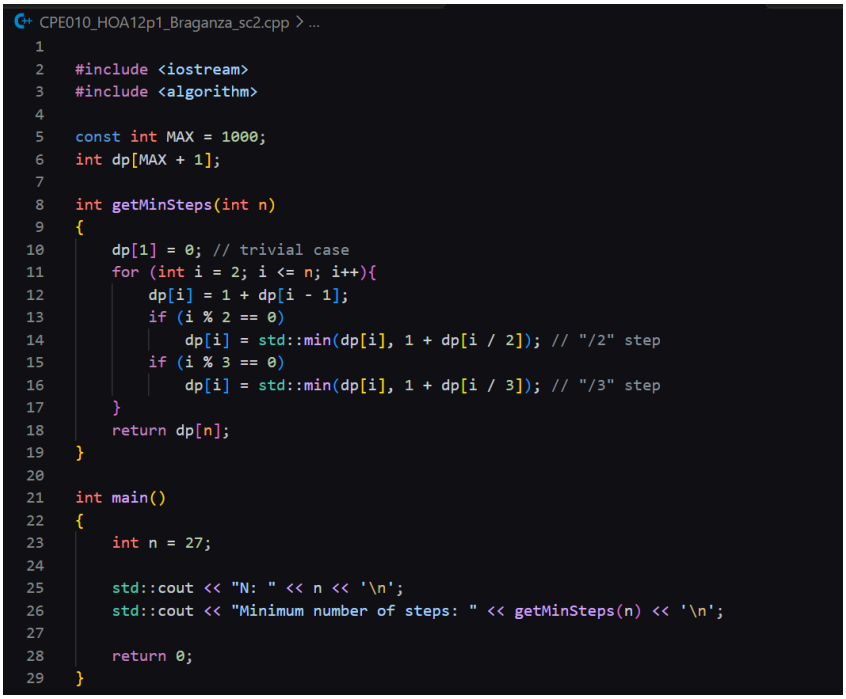
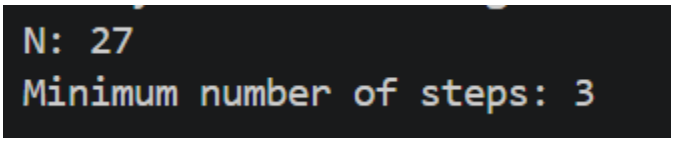
Screenshot of the code	
Output of the code	
Analysis	This bottom to up algorithm operates by looping and building a table array from the lowest number up to N multiple times until it hits the final input number and then comparing the paths from subtracting 1 or dividing by 2 or 3 and selecting the minimum count until it returns the total number of steps but the answers inside of the array were already in the correct saved position or completed. For the input value of 27, it builds up the table sequentially until it finds the quickest way to hit 1, which gives a final output of 3 minimum steps on the console screen. It takes almost the exact same time no matter how big the input number is because the loop fills up every slot in the table in a steady forward order.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO B

Task Title & Objective:

<u>Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies</u>

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description: Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller “subproblems” until the solution becomes trivial.

Deliverables:

- Pseudocode (+ Solving Problem by Hand)
- Working C++ Code
- Analysis of working code
- Screenshots of demonstration

Pseudocode (+ Solving Problem by Hand)

```
START
FUNCTION countPaths(matrix, i, j, cost)
   IF i >= max_rows OR j >= max_cols THEN
      RETURN 0
   IF cost < matrix[i][j] THEN
      RETURN 0
   IF memo[i][j][cost] != -1 THEN
      RETURN memo[i][j][cost]

   cost = cost - matrix[i][j]

   IF i == max_rows - 1 AND j == max_cols - 1 THEN
      IF cost == 0 THEN
         RETURN 1
      ELSE
         RETURN 0

   int downPaths = countPaths(i + 1, j, cost)
   int rightPaths = countPaths(i, j + 1, cost)

   memo[i][j][cost + matrix[i][j]] = downPaths + rightPaths
   RETURN memo[i][j][cost + matrix[i][j]]

END FUSET

   cost = 27

   DISPLAY the given cost
   DISPLAY the number of valid paths from (0, 0)NCTION

STOP
```

Working C++ Code	CPE010_HOA12p1_Braganza_sc3.cpp > ...
	`#include <iostream>`
	`#include <cstring>`
	
	`const int M = 4;`
	`const int N = 4;`
	`const int maxC = 100;`
	
	`int matrix[M][N] = {`
	`{4, 7, 1, 6},`
	`{6, 7, 3, 9},`
	`{3, 8, 1, 2},`
	`{7, 1, 7, 3}`
	`};`
	
	`int memo[M][N][maxC + 1];`
	
	`int countPaths(int i, int j, int cost)`
	`{`
	`if (i >= M || j >= N) return 0; // outside the matrix`
	`if (cost < matrix[i][j]) return 0; // cost exceeded`
	`if (memo[i][j][cost] != -1)`
	`return memo[i][j][cost]; // we have solved it already`
	`cost -= matrix[i][j]; // subtract current cell cost`
	`if (i == M - 1 && j == N - 1)`
	`return (cost == 0) ? 1 : 0; // destination cell`
	`int r = countPaths(i + 1, j, cost); // move down`
	`r += countPaths(i, j + 1, cost); // move right`
	`memo[i][j][cost + matrix[i][j]] = r; // saving the result`
	
	`return r;`
	`}`
	
	`int main()`
	`{`
	`std::memset(memo, -1, sizeof(memo));`
	
	`int cost = 27;`
	
	`std::cout << "Given Cost: " << cost << '\n';`
	`std::cout << "Number of Paths: "<< countPaths(0, 0, cost) << '\n';`
	
	`return 0;`
	`}`

Output of the code	Given Cost: 27 Number of Paths: 2

Analysis of working code (Bal)

This supplementary, this algorithm works by checking the matrix moves multiple times using recursion to branch down and right until it hits the final bottom corner cell and then comparing the paths and adding them up while saving each distinct state into a three dimensional memory array until it returns the total path count but the answers inside of the array were already in the correct saved position or completed. For the input target cost value of 27, it explores the adjacent routes sequentially by subtracting cell values from the current balance. It runs very quickly because the function always checks the memory array first to see if it already calculated that exact location and remaining cost, which gives a stable time complexity of O(M x N x cost) and keeps it faster and more dependable than plain recursion when handling large grid data or large target values.

Demonstration of the code Bal

The program starts running inside the main function by setting up a triple loop to fill the entire three dimensional memory array with negative ones to show that no steps have been tracked yet. After that, it initializes the matrix values and sets the target cost budget to 27 before calling the path function at the very first cell location of zero and zero. As the math process begins, the algorithm steps through the grid cells one by one from start to end by only branching down or right, calculating the combined path sums like 11 at index zero and one or 10 at index one and zero. If the calculation hits a matrix boundary wall or drops the remaining budget below zero, it shuts down that branch early, but if it successfully reaches the bottom right corner with a remaining cost of zero, it adds 1 to the success count. During this whole journey, the code checks the memory table array at each cell to completely avoid repeating the math for any path it has already solved. At the very end of the calculation, the program finds that a target cost of 25 produces zero paths, but changing the target to 27 successfully isolates two specific valid routes and prints out the final path count of 2 on the terminal screen.

The program executes from start to end by running through two separate step-by-step path sequences to check how cell values accumulate across the grid positions before reaching the destination coordinate. In the first calculation line, the program starts at location zero and zero with a value of 4, then moves right to step one to add 7 making 11, moves down to step two to add 7 making 18, moves right to step three to add 3 making 21, moves down to step four to add 1 making 22, moves right to step five to add 2 making 24, and finally moves down at the end step to add 3 to hit the goal of 27. In the second calculation line, the program takes a different path from the same start by moving down to step one to add 6 making 10, down to step two to add 3 making 13, right to step three to add 8 making 21, and then it merges onto the exact same final tracking moves for steps four, five, and six to hit the target value of 27. Because both calculation sequences successfully reduce the target budget from 27 down to exactly zero at the bottom-right corner, the function registers each sequence as a valid route and prints out the final path count of 2 on the terminal window.

Process Line 1: Upper Route Calculation Sequence

Start	Step 1	Step 2	Step 3	Step 4	Step 5	End
(0,0)	(0,1)	(1,0)	(1,2)	(2,2)	(2,3)	(3,3)
4	4+7 = 11	11+7 = 18	18+3 = 21	21+1 = 22	22+2 = 24	24+3 = 27

Process Line 2: Lower Route Calculation Sequence

Start	Step 1	Step 2	Step 3	Step 4	Step 5	End
(0,0)	(1,0)	(2,0)	(2,1)	(2,2)	(2,3)	(3,3)
4	4+6 = 10	10+3 = 13	13+8 = 21	21+1= 22	22+2 = 24	24+3 = 27

Analysis of working code (Braganzal)

This program implements matrix[M][N] and memo[M][N][maxC+1] arrays in order to track the original 4x4 matrix and the previous calculated values. With respect to the countPaths() method, the restrictions on boundaries and costs make sure that no invalid path is examined by the program. The countPaths(i+1, j, cost) and countPaths(i, j+1, cost) can be used to examine the possible movement downward and rightward while saving values using memoization technique.

At the beginning of the main() function, std::memset(memo,-1,sizeof(memo)) initializes the memoization table whereas int cost=27; specifies the value of cost. After calling the countPaths(0,0,cost), the program reveals two paths that meet the specified criteria. It can be seen that the program has been able to use recursion and memoization techniques quite efficiently to solve this problem.

Analysis of working code (Braganza)

The program finds 2 paths whose total cost is 27.
PATH 1
It moves down, down, right, right, right, down

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

Cell values: 4 → 6 → 3 → 8 → 1 → 2 → 3
4+6+3+8+1+2+3
= 27

The remaining cost is calculated at every cell:

Cell value	Calculation	Remaining cost
4	27 − 4	23
6	23 − 6	17
3	17 − 3	14
8	14 − 8	6

1	6 − 1	5
2	5 − 2	3
3	3 − 3	0

PATH 2

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

Cell values: 4 → 7 → 7 → 3 → 1 → 2 → 3
4+7+7+3+1+2+3
=27

Cell value	Calculation	Remaining cost
4	27 − 4	23
7	23 − 7	16
7	16 − 7	9
3	9 − 3	6
1	6 − 1	5
2	5 − 2	3
3	3 − 3	0

Analysis of the supplementary activity (Bal)

The supplementary activity applies this path-counting algorithm onto a four by four matrix to evaluate how many paths can reach the destination cell under a specific budget restriction. When running the initial program layout with a target cost parameter of 25, the code discovers that there are zero valid paths

available because none of the grid choices add up perfectly to that exact amount. To address this and verify that the program functions correctly, the user can manually solve the table values to find a valid route and then alter the target parameter to 27. Changing this target value allows the code to successfully discover and display exactly 2 matching path trajectories through the matrix cells. This adjustment confirms that using a three-dimensional table effectively avoids duplicate tracking, giving a reliable constant time check for every cell combination.

Conclusion [Bal]

This activity demonstrates how to find the total number of valid paths through a grid when a specific target cost must be met perfectly. The implementation teaches how to combine top-down recursion with a three-dimensional array to remember past calculations for specific coordinates and remaining path allowance. Storing these intermediate paths prevents the system from recalculating identical states repeatedly, which shows why dynamic programming is highly valuable for keeping grid-based traversal problems running efficiently and also teaches how memoization functions to store the results of expensive function calls so the program does not have to do the exact same calculations again. It demonstrates that breaking down a big grid problem into overlapping smaller subproblems and solving each only once can completely change the execution speed from slow and exponential to a fast and instant linear runtime. By using a combination of top-down recursion and caching values inside a three-dimensional table array, the logic ensures that if the system ever encounters an already solved cell and cost combination, it can just return the stored value immediately. This approach shows a practical way to manage path allowance across a grid, proving how saving and keeps data organized and cuts down on repetitive math operations.

Conclusion [Braganza]

After performing this hands-on activity, I understood that there are various algorithmic techniques like recursion, brute force, backtracking, greedy algorithm, divide and conquer, and dynamic programming. I understood the ways in which these algorithmic techniques can be employed for solving various problems and how memoization and dynamic programming from the bottom up can help in avoiding the computation that is redundant. In the additional task, I have implemented recursion and memoization to find out the number of paths possible in the matrix at a particular cost. The program correctly identified two paths worth a total of 27.

Overall, through this activity, I have learnt more about the importance of having the right approach when trying to solve a problem since this will help me make the problem simpler and hence enable me to solve the problem. While I still need to learn how to use these approaches in solving various problems, this process was helpful in consolidating my knowledge of the lesson.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini, QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, code assistance and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI helped us understand the lessons, explain the code, and organize the analysis. However, we still used it to reviewed the work, checked the calculations, and completed the required tasks myself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Analysis of working code (Bal)

This supplementary, this algorithm works by checking the matrix moves multiple times using recursion to branch down and right until it hits the final bottom corner cell and then comparing the paths and adding them up while saving each distinct state into a three dimensional memory array until it returns the total path count but the answers inside of the array were already in the correct saved position or completed. For the input target cost value of 27, it explores the adjacent routes sequentially by subtracting cell values from the current balance. It runs very quickly because the function always checks the memory array first to see if it already calculated that exact location and remaining cost, which gives a stable time complexity of O(M x N x cost) and keeps it faster and more dependable than plain recursion when handling large grid data or large target values.

Analysis of working code (Bal)

This supplementary, this algorithm works by checking the matrix moves multiple times using recursion to branch down and right until it hits the final bottom corner cell and then comparing the paths and adding them up while saving each distinct state into a three dimensional memory array until it returns the total path count but the answers inside of the array were already in the correct saved position or completed. For the input target cost value of 27, it explores the adjacent routes sequentially by subtracting cell values from the current balance. It runs very quickly because the function always checks the memory array first to see if it already calculated that exact location and remaining cost, which gives a stable time complexity of O(M x N x cost) and keeps it faster and more

dependable than plain recursion when handling large grid data or large target values.